메모리풀 성능 튜닝 가이드

개요

CometBFT 메모리풀의 성능은 전체 블록체인 네트워크의 처리량과 지연시간에 직접적인 영향을 미칩니다. 이 문서는 메모리풀 관련 설정 옵션들과 그 성능 영향을 분석하고, 다양한 환경에서의 최적화 방법과 일반적인 문제 해결 방법을 제시합니다.

성능 지표 및 모니터링

핵심 성능 메트릭

CometBFT는 메모리풀 성능을 모니터링하기 위한 다양한 메트릭을 제공합니다:

```
type Metrics struct {

// 용량 관련 메트릭

Size metrics.Gauge // 전체 트랜잭션 개수

SizeBytes metrics.Gauge // 전체 메모리 사용량

LaneSize metrics.Gauge // 레인별 트랜잭션 개수

LaneBytes metrics.Gauge // 레인별 메모리 사용량

// 성능 관련 메트릭

TxLifeSpan metrics.Histogram // 트랜잭션 생존 시간

TxSizeBytes metrics.Histogram // 트랜잭션 크기 분포

// 처리 결과 메트릭

FailedTxs metrics.Counter // 검증 실패 트랜잭션
```

```
RejectedTxs metrics.Counter // 용량 부족으로 거부된 트랜잭션
EvictedTxs metrics.Counter // 재검증에서 제거된 트랜잭션
// 효율성 메트릭
AlreadyReceivedTxs metrics.Counter // 중복 수신 트랜잭션
RecheckTimes metrics.Counter // 재검증 횟수
RecheckDurationSeconds metrics.Gauge // 재검증 소요 시간
}
```

메트릭 수집 설정

```
# config.toml
[instrumentation]
# Prometheus 메트릭 활성화
prometheus = true
prometheus_listen_addr = ":26660"

# 메트릭 네임스페이스
namespace = "cometbft"

# 최대 연결 수
max_open_connections = 3
```

성능 모니터링 대시보드

```yaml
# Prometheus 쿼리 예제
queries:
# 메모리풀 사용률
- name: "mempool_utilization"
query: "cometbft_mempool_size / 5000 * 100"
# 평균 트랜잭션 생존 시간
- name: "avg_tx_lifetime"
query: "rate(cometbft_mempool_tx_life_span_sum[5m]) / rate(cometbft_mempool_tx_l
# 트랜잭션 처리율
- name: "tx_throughput"
query: "rate(cometbft_mempool_evicted_txs[5m]) + rate(cometbft_mempool_failed_tx
# 캐시 히트율
- name: "cache_hit_rate"
query: "rate(cometbft_mempool_already_received_txs[5m]) / (rate(cometbft_mempool
```

설정 옵션 및 성능 영향

기본 용량 설정

```
[mempool]
# 메모리풀 타입 선택
type = "flood" # "flood" 또는 "nop"
```

```
# 최대 트랜잭션 개수
size = 5000

# 전체 메모리풀 최대 크기 (바이트)
max_txs_bytes = 67108864 # 64MB

# 단일 트랜잭션 최대 크기 (바이트)
max_tx_bytes = 1048576 # 1MB
```

성능 영향 분석:

설정	증가 시 효과	감소 시 효과	권장값
`size`	더 많은 트랜잭션 보관, 메모리 사용량 증가	빠른 처리, 메모리 절약	5000-10000
`max_txs_bytes`	큰 트랜잭션 허용, 메모리 사용량 증가	메모리 절약, 큰 트랜잭션 거부	64-128MB
`max_tx_bytes`	큰 단일 트랜잭션 허용	네트워크 효율성 향상	1-4MB

캐시 관련 설정

```
[mempool]
# 캐시 크기 (트랜잭션 개수)
cache_size = 10000

# 무효한 트랜잭션 캐시 유지 여부
```

```
keep-invalid-txs-in-cache = false
```

최적화 가이드:

```
# 캐시 히트율 확인
curl -s http://localhost:26660/metrics | grep mempool_already_received_txs

# 캐시 크기 조정 기준
# 히트율 < 30%: 캐시 크기 증가 고려
# 히트율 > 80%: 캐시 크기 적절
# 메모리 부족: 캐시 크기 감소
```

재검증 설정

```
[mempool]
# 블록 커밋 후 재검증 활성화
recheck = true

# 재검증 타임아웃
recheck_timeout = "1s"
```

성능 트레이드오프:

- **recheck = true**: 정확성 향상, CPU 사용량 증가
- **recheck = false**: 성능 향상, 무효한 트랜잭션 누적 가능

네트워크 최적화 설정

```
[mempool]
# 트랜잭션 브로드캐스트 활성화
broadcast = true

# DOG 프로토콜 (대역폭 최적화)
dog_protocol_enabled = false
dog_target_redundancy = 1.0
dog_adjust_interval = "1s"

# 실험적 기능: 가십 연결 제한
experimental_max_gossip_connections_to_persistent_peers = 0
experimental_max_gossip_connections_to_non_persistent_peers = 0
```

환경별 최적화 전략

1. 고성능 밸리데이터 노드

```
[mempool]
type = "flood"
size = 20000
max_txs_bytes = 268435456 # 256MB
```

```
max_tx_bytes = 4194304 # 4MB

cache_size = 50000

recheck = true

recheck_timeout = "2s"

broadcast = true

keep-invalid-txs-in-cache = false



# DOG 프로토콜 활성화로 대역폭 최적화

dog_protocol_enabled = true

dog_target_redundancy = 0.8

dog_adjust_interval = "500ms"
```

하드웨어 권장사항:

- CPU: 16+ 코어
- RAM: 32GB+
- 네트워크: 1Gbps+

2. 일반 풀노드

```
[mempool]

type = "flood"

size = 5000

max_txs_bytes = 67108864 # 64MB

max_tx_bytes = 1048576 # 1MB
```

```
cache_size = 10000

recheck = true

recheck_timeout = "1s"

broadcast = true

keep-invalid-txs-in-cache = false



# 가십 연결 제한으로 대역폭 절약

experimental_max_gossip_connections_to_non_persistent_peers = 10
```

하드웨어 권장사항:

- CPU: 4-8 코어
- RAM: 8-16GB
- 네트워크: 100Mbps+

3. 경량 노드 (RPC 전용)

```
[mempool]

type = "nop" # 메모리풀 비활성화
```

또는 최소 설정:

```
[mempool]

type = "flood"

size = 1000

max_txs_bytes = 16777216 # 16MB
```

```
max_tx_bytes = 524288 # 512KB

cache_size = 2000

recheck = false

broadcast = false
```

4. 개발/테스트 환경

```
[mempool]

type = "flood"

size = 1000

max_txs_bytes = 16777216 # 16MB

max_tx_bytes = 1048576 # 1MB

cache_size = 1000

recheck = true

recheck_timeout = "500ms"

broadcast = true

keep-invalid-txs-in-cache = true # 디버깅 용이성
```

성능 최적화 기법

1. ABCI 애플리케이션 최적화

CheckTx 성능 향상

```go
// 빠른 사전 검증
func (app *MyApp) CheckTx(req abci.RequestCheckTx) abci.ResponseCheckTx {
tx := req.Tx
// 1. 기본 형식 검증 (빠름)
if  len(tx) == 0 || len(tx) > maxTxSize {
return abci.ResponseCheckTx{Code: 1, Log: "invalid tx size"}
}
// 2. 서명 검증 (상대적으로 느림)
if !app.verifySignature(tx) {
return abci.ResponseCheckTx{Code: 2, Log: "invalid signature"}
}
// 3. 상태 기반 검증 (가장 느림)
if !app.validateAgainstState(tx) {
return abci.ResponseCheckTx{Code: 3, Log: "invalid state"}
}
return abci.ResponseCheckTx{Code: 0, GasWanted: app.estimateGas(tx)}
}
```

레인 활용

```go
func (app *MyApp) CheckTx(req abci.RequestCheckTx) abci.ResponseCheckTx {
tx := parseTx(req.Tx)
// 트랜잭션 타입에 따른 레인 할당
```

```
var laneId string
switch tx.Type {
case "transfer":
laneId = "default"
case "stake":
laneId = "staking"
case "governance":
laneId = "gov"
}
return abci.ResponseCheckTx{
Code: 0,
LaneId: laneId,
GasWanted: app.estimateGas(tx),
}
}
```

2. 네트워크 최적화

피어 연결 관리

```
[p2p]
# 최대 피어 연결 수
max_num_inbound_peers = 40
max_num_outbound_peers = 10
```

```
# 연결 유지 설정
persistent_peers_max_dial_period = "0s"
flush_throttle_timeout = "100ms"

# 메시지 큐 크기
recv_rate = 5120000 # 5MB/s
send_rate = 5120000 # 5MB/s
```

DOG 프로토콜 튜닝

```
// DOG 프로토콜 설정 최적화
type redundancyControl struct {
targetRedundancy  float64  // 목표 중복도
adjustInterval time.Duration
currentRedundancy atomic.Value
}

func (rc *redundancyControl) optimize() {
// 네트워크 상황에 따른 동적 조정
if  rc.getCurrentRedundancy() > rc.targetRedundancy*1.2 {
rc.reduceRoutes()
} else  if  rc.getCurrentRedundancy() < rc.targetRedundancy*0.8 {
```

```
rc.increaseRoutes()
}
}
```

3. 메모리 최적화

가비지 컬렉션 튜닝

```
# Go 런타임 환경 변수
export GOGC=100 # GC 목표 백분율
export GOMEMLIMIT=8GiB # 메모리 제한
export GOMAXPROCS=8 # 최대 프로세서 수
```

메모리 풀링

```
// 메시지 객체 재사용
var txMessagePool = sync.Pool{
New: func() interface{} {
return &protomem.Txs{Txs: make([][]byte, 0, 100)}
},
}

func getTxMessage() *protomem.Txs {
msg := txMessagePool.Get().(*protomem.Txs)
msg.Txs = msg.Txs[:0] // 슬라이스 재설정
```

```
return msg
}

func putTxMessage(msg *protomem.Txs) {
if cap(msg.Txs) < 1000 { // 너무 큰 슬라이스는 풀에 반환하지 않음
txMessagePool.Put(msg)
}
}
```

일반적인 성능 문제 및 해결방법

1. 메모리풀 포화 상태

증상:

- `RejectedTxs` 메트릭 급증
- 트랜잭션 처리 지연
- 메모리 사용량 지속적 증가

진단:

```
# 메모리풀 상태 확인
curl -s http://localhost:26657/num_unconfirmed_txs

# 메트릭 확인
```

```
curl -s http://localhost:26660/metrics | grep -E "(mempool_size|mempool_reje
```

해결방법:

```
# 1. 용량 증가
[mempool]
size = 10000
max_txs_bytes = 134217728 # 128MB

# 2. 재검증 최적화
recheck_timeout = "2s"

# 3. 캐시 크기 증가
cache_size = 20000
```

2. 높은 재검증 오버헤드

증상:

- `RecheckDurationSeconds` 메트릭 높음
- 블록 커밋 후 지연 발생
- CPU 사용률 스파이크

해결방법:

```
# 재검증 비활성화 (주의: 정확성 트레이드오프)
[mempool]
```

```
recheck = false


# 또는 타임아웃 단축
recheck_timeout = "500ms"
```

ABCI 애플리케이션 최적화:

```
func (app *MyApp) CheckTx(req abci.RequestCheckTx) abci.ResponseCheckTx {
// 재검증 시 빠른 경로 제공
if req.Type == abci.CheckTxType_Recheck {
return app.fastRecheck(req.Tx)
}
return app.fullCheck(req.Tx)
}
```

3. 네트워크 대역폭 과부하

증상:

- 높은 네트워크 사용률
- 트랜잭션 전파 지연
- 피어 연결 불안정

해결방법:

```
[mempool]
```

```
# DOG 프로토콜 활성화
dog_protocol_enabled = true
dog_target_redundancy = 0.5 # 중복도 감소


# 가십 연결 제한
experimental_max_gossip_connections_to_non_persistent_peers = 5
```

4. 메모리 누수

증상:

- 메모리 사용량 지속적 증가
- GC 빈도 증가
- 시스템 성능 저하

진단 및 해결:

```
# 메모리 프로파일링
go tool pprof http://localhost:6060/debug/pprof/heap


# 메모리풀 플러시 (임시 해결책)
curl -X POST http://localhost:26657/unsafe_flush_mempool
```

코드 레벨 최적화:

```
// 적절한 리소스 해제
func (mem *CListMempool) RemoveTxByKey(txKey types.TxKey) error {
```

```
// ... 기존 코드 ...

// 메모리 누수 방지

elem.DetachPrev()

elem.Value = nil  // 참조 해제

return  nil

}
```

벤치마킹 및 성능 테스트

부하 테스트 스크립트

```
#!/bin/bash

# mempool_load_test.sh


# 테스트 매개변수

TPS=${1:-1000}  # 초당 트랜잭션 수

DURATION=${2:-60}  # 테스트 지속 시간 (초)

TX_SIZE=${3:-250}  # 트랜잭션 크기 (바이트)


echo  "Starting mempool load test: ${TPS} TPS for ${DURATION}s"


# 백그라운드에서 트랜잭션 전송

for  i  in $(seq  1  $DURATION);  do
```

```
for j in $(seq 1 $TPS); do
# 랜덤 트랜잭션 생성 및 전송
curl -s -X POST http://localhost:26657/broadcast_tx_async \
-d "{\"tx\":\"$(openssl rand -hex $TX_SIZE)\"}" > /dev/null &
done
sleep 1
# 진행 상황 출력
echo "Progress: $i/$DURATION seconds"
# 메모리풀 상태 확인
curl -s http://localhost:26657/num_unconfirmed_txs | jq '.result.total'
done

wait
echo "Load test completed"
```

성능 측정 스크립트

```
#!/usr/bin/env python3
# mempool_metrics.py

import requests
import time
import json
```

```
from datetime import datetime


def  collect_metrics():

"""메모리풀 메트릭 수집"""

try:

# Prometheus 메트릭 수집

response = requests.get('http://localhost:26660/metrics')

metrics = {}

for line in response.text.split('\n'):

if line.startswith('cometbft_mempool_'):

parts = line.split(' ')

if  len(parts) == 2:

metric_name = parts[0]

metric_value = float(parts[1])

metrics[metric_name] = metric_value

return metrics

except Exception as e:

print(f"Error collecting metrics: {e}")

return {}


def  monitor_performance(duration=300, interval=5):

"""성능 모니터링"""

print(f"Monitoring mempool performance for {duration} seconds...")
```

```
start_time = time.time()

while time.time() - start_time < duration:

metrics = collect_metrics()

timestamp = datetime.now().isoformat()

# 주요 메트릭 출력

print(f"[{timestamp}]")

print(f" Size: {metrics.get('cometbft_mempool_size', 0)}")

print(f" Size Bytes: {metrics.get('cometbft_mempool_size_bytes', 0)}")

print(f" Failed Txs: {metrics.get('cometbft_mempool_failed_txs', 0)}")

print(f" Rejected Txs: {metrics.get('cometbft_mempool_rejected_txs', 0)}")

print(f" Cache Hit Rate: {calculate_cache_hit_rate(metrics):.2%}")

print()

time.sleep(interval)


def  calculate_cache_hit_rate(metrics):

"""캐시 히트율 계산"""

already_received = metrics.get('cometbft_mempool_already_received_txs', 0)

total_requests = already_received + metrics.get('cometbft_mempool_size', 0)

if total_requests == 0:

return  0.0

return already_received / total_requests


if __name__ == "__main__":
```

```
monitor_performance()
```

모범 사례 요약

1. 설정 최적화 체크리스트

- [ ] 하드웨어 사양에 맞는 메모리풀 크기 설정
- [ ] 네트워크 환경에 따른 캐시 크기 조정
- [ ] 애플리케이션 특성에 맞는 재검증 설정
- [ ] DOG 프로토콜 활성화 고려
- [ ] 가십 연결 수 제한 설정

2. 모니터링 체크리스트

- [ ] Prometheus 메트릭 수집 활성화
- [ ] 핵심 성능 지표 대시보드 구성
- [ ] 알림 임계값 설정
- [ ] 정기적인 성능 테스트 수행

3. 운영 체크리스트

- [ ] 로그 레벨 적절히 설정
- [ ] 메모리 사용량 정기 모니터링
- [ ] 네트워크 대역폭 사용량 추적
- [ ] 백업 및 복구 절차 수립

참고 자료

- 메모리풀 개요
- 트랜잭션 처리 과정
- 캐싱 및 관리 전략
- ABCI 성능 최적화
- P2P 네트워크 튜닝